

GPW WATS 3.02

Internet Data Distribution System.

Date: 25.03.2024 | Version: GPW0.62

Contents

1.	Disclaimer	3
2.	Preface.....	4
2.1.	Target Audience.....	4
2.2.	Document's Purpose	4
2.3.	Associated Documents.....	4
3.	Document History.....	5
4.	Overview to IDDS	6
4.1.	Connectivity	6
5.	Introduction to REST	7
5.1.	Using Rest API.....	7
6.	Introduction to Streaming	9
6.1.	Topics.....	9
6.2.	Client Configuration.....	9

1. DISCLAIMER

This document is for information purposes only and does not form any part of contractual documentation.

Reasonable care has been taken to ensure details contained within are accurate and not misleading at the time of publication. Warsaw Stock Exchange is not responsible for any errors or omissions contained in this document.

Warsaw Stock Exchange reserves the right to treat information contained in this document subject to later change without prior notice.

This document contains confidential information to Warsaw Stock Exchange and may not be reproduced, disclosed, or used in whole or part, in any manner, without prior written consent from the owner of this document. Information included in this document shall be maintained and exercised with adequate security measures necessary to protect confidential information from unauthorized access or disclosure.

In case of sections of documentation at a **High** level work progress according to the current version of *GPW WATS Advancement of Documentation*, Warsaw Stock Exchange will endeavor to limit changes to these sections of documents to those related to:

1. correcting errors in the documentation or in the software;
2. clarification of the documentation content or removing ambiguity;
3. implementation of approved change requests or;
4. regulatory changes.

2. PREFACE

This section describes the basic information about GPW WATS Internet Data Distribution System. You can learn about the target audience, the document purpose and all the necessary documents you should read in relation to this Specification.

2.1. TARGET AUDIENCE

This document is dedicated to entities, such as Independent Software Vendors who produce software integrated with GPW.

2.2. DOCUMENT'S PURPOSE

This document describes how to obtain OAuth Token and interact with our Rest and Stream APIs.

2.3. ASSOCIATED DOCUMENTS

Please check the following documents to learn more about IDDS:

- GPW WATS 3.03 Streaming Messages for IDDS
- GPW WATS 3.04 Rest Messages for IDDS

3. DOCUMENT HISTORY

Version	Date	Description
0.61	29.02.2024	Initial publication of the document.
0.62	25.03.2024	Publication of v0.62. No changes in the document.

4. OVERVIEW OF IDDS

IDDS is an Internet Data Distribution System. It is an alternative data distribution channel to WATS DDS service that operates online.

4.1. CONNECTIVITY

This system enables to access data by the use of two protocols:

- Rest API
- Streaming API (Kafka protocol)

4.1.1. AUTHENTICATION

IDDS uses OAuth standard for Authentication and Authorization. Before logging into IDDS, GPW has to provide you with the necessary data. To interact with IDDS you need to receive from GPW:

- connection URL
- client id
- secret.

To receive OAuth token, send a following request to authorization server, filling the request with the data provided by GPW:

Example request:

```
curl -s -X POST 'http://[host]:[port]]/realms/[realm]/protocol/openid-connect/token' \  
--header 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'grant_type=client_credentials' \  
--data-urlencode 'client_id=[clientId]' \  
--data-urlencode 'client_secret=[secret]'
```

Example response - HTTP Status 200

```
{  
  "access_token": "eyJh...",  
  "expires_in": 300,  
  "refresh_expires_in": 0,  
  "token_type": "Bearer",  
  "not-before-policy": 0,  
  "scope": "profile email",  
  ...  
}
```

In the response, you receive all the necessary parameters of the access_token. The access_token field contains a token that should be used for authentication both in Rest and Streaming API. It is important to stress that the same token is used for both of the protocols.

5. INTRODUCTION TO REST

Rest API shares data using HTTP protocol secured with SSL. To obtain data you need to use your token (to see how to generate your token, check *4.1.1 Authentication* section) and an URL of the data package you would like to access. The data received with Rest API is especially useful while Reference Data is required, as you need to repeat your request to obtain the most up to date data.

AVAILABLE URLS:

Reference Data

- /api/v1/reference/{date}/market-structures
- /api/v1/reference/{date}/calendar
- /api/v1/reference/{date}/markets/{mic}/instruments/list
- /api/v1/reference/{date}/markets/{mic}/instruments
- /api/v1/reference/{date}/instruments
- /api/v1/reference/{date}/indexes
- /api/v1/reference/{date}/markets/{mic}/accrued-interest-tables
- /api/v1/reference/{date}/markets/{mic}/kids

Market Data

- /api/v1/snapshot/instrument/{id}
- /api/v1/snapshot/index/{id}
- /api/v1/summary/instrument
- /api/v1/summary/index

Common

- /api/v1/schema/avro

For the complete instructions and description of the received data for each URL, check *GPW WATS 16.03 Rest Messages for IDDS*.

5.1. USING REST API

1. Request for access token (described in *4.1.1 Authentication* subsection).
2. Request with token:

Example request

```
curl -X GET "https://[host]:[port]/[rest endpoint]" \  
-H "Authorization: Bearer [Your-Access-Token]"
```

Example response

When you make a successful GET request to the `/api/v1/reference/{date}/market-structures` endpoint, the response includes a collection of `MarketStructureView` objects. Here's an example of what the response might look like:

```
[
  {
    "id": 0,
    "mic": "string",
    "name": "string",
    "marketModelType": "NOTAPPLICABLE",
    "children": [
      {
        "id": 0,
        "mic": "string",
        "name": "string",
        "marketModelType": "NOTAPPLICABLE",
        "children": [
          ...
        ]
      }
    ]
  }
]
```

6. INTRODUCTION TO STREAMING

Streaming API shares data using Kafka protocol. Clients are authenticated using Kafka SASL OAUTHBEARER mechanism. You receive live data as long as you are subscribed to one on the following data topics.

To connect, you have to configure your Kafka Client via JAAS configuration and use Strimzi Kafka OAuth extension.

6.1. TOPICS

Common:

- avro_schema
- fd_refdata
- fd_refdata_nextsession

Full Data Live:

- fd_equities
- fd_structures
- fd_derivatives
- fd_bonds
- fd_indexes

BBO Live:

- bbo_equities
- bbo_structures
- bbo_derivatives
- bbo_bonds
- bbo_indexes

BBO Delayed:

- bbo_equities_delayed
- bbo_structures_delayed
- bbo_derivatives_delayed
- bbo_bonds_delayed
- bbo_indexes_delayed

6.2. CLIENT CONFIGURATION

The example below is an example of configuring Kafka client to connect to IDDS. The example is based on a console client distributed with Kafka.

Building Strimzi Kafka OAuth Extension

Kafka doesn't support OAuth token authentication, so it is crucial to install the following extension.

1. Pull Strimzi Kafka OAuth extension form: [Strimzi Kafka OAuth extension](#).
2. Build the project with steps described here [Building Strimzi Kafka](#).
3. Copy built artifacts listed here ([Artifacts](#)) to your directory.

Example JAAS Configuration

Remember to replace your **clientId**, **secret** and **connection URL** in the indicated places.

```
security.protocol=SASL_PLAINTEXT
sasl.mechanism=OAUTHBEARER
sasl.jaas.config=org.apache.kafka.common.security.oauthbearer.OAuthBearerLogin
Module required \
  oauth.client.id="your client id" \
  oauth.client.secret="your secret" \
  oauth.token.endpoint.uri="https://[host]:[port]]/realms/[realm]/protocol/openid-connect/token" ;
sasl.login.callback.handler.class=io.strimzi.kafka.oauth.client.JaasClientOAuthLoginCallbackHandler
```

WARNING! IN THE PRODUCTION SYSTEM, WE USE THE SASL_SSL PROTOCOL.

Receive Avro Schema

```
export CLASSPATH=/path to your strimzi plugin/*:$CLASSPATH
bin/kafka-console-consumer.sh --bootstrap-server [kafka broker host]:[kafka broker port] --topic avro_schema \
  --from-beginning --consumer.config your-jaas-config.properties
```

6.2.1. READ TOPIC WITH AVRO DESERIALIZATION

Then you are successfully connected, authenticated, authorized and received avro schema. Now you can consume data topic and deserialize it.